

LLM Fundamentals

LLM-Agents

Presented by: **Aleksandr Algazinov**



HSE University



None of this is thinking.

Every line of it is one token, and then the next, and then the next. Today is about what that sentence actually means — and about what it lets you.

Motivation

Three things you have already seen a model do, and cannot yet explain.

1 • IT MISCOUNTS LETTERS

Ask how many **r**'s are in "strawberry". It may well get that wrong.

2 • THE SAME PROMPT GIVES DIFFERENT ANSWERS

Run it twice, get two outputs. Sometimes that is wanted. Often it is not.

3 • A LONG DOCUMENT COSTS MORE THAN A SHORT ONE

Not proportionally to words, and not to characters. To something else.

Each is answered by one block of this lecture.

It does not think. It generates.

A language model has exactly one move: given everything so far, produce the next token. Then do it again.

Everything you have heard called reasoning, planning or thinking is a **structure over that loop** — not an escape from it.

That is not a criticism of the technology. It is the specification.

Why that makes you better at prompting

- 1 Tokens** — it never received letters. Character-level tasks are structurally hard, so give it a tool rather than a firmer instruction.
- 2 Attention** — every token reweights every later one. Irrelevant context is not free padding; it is active interference.
- 3 Next-token prediction** — it cannot un-say a token. Anything it must weigh has to be generated *before* the thing it decides.
- 4 Generation parameters** — a format is a constraint on a distribution. State the schema before the content and you put the model where that schema lives.

Four consequences of mechanism. The rest of the lecture is where they come from.

Tokens and Embeddings

What the model actually receives

The model never sees characters

Your text is split into **tokens** before the model sees anything.



three consequences, all of them yours to live with

It cannot count letters

there is no 'r' in its input — only pieces

The context limit is in tokens

not words, not characters

The bill is in tokens

formatting and whitespace change the count

DISCUSSION

**You have a page of text and a model that only accepts integers.
How do you split it?**

How the split is chosen

Byte-Pair Encoding: start from characters, merge the most frequent pair, repeat.

Merge the most frequent adjacent pair. Repeat.

corpus: low ×5 lower ×2 newest ×6 widest ×3

START · characters

l o w · l o w e r · n e w e s t · w i d e s t

e + s → es (9×)

l o w · l o w e r · n e w **e s** t · w i d **e s** t

es + t → est (9×)

l o w · l o w e r · n e w **e s t** · w i d **e s t**

l + o → lo (7×)

l o w · **l o** w e r · n e w **e s t** · w i d **e s t**

lo + w → low (7×)

l o w · **l o w** e r · n e w **e s t** · w i d **e s t**

STOP at the vocabulary budget.

"low" → 1 token
a common word collapses

"strawberry" → str | aw | berry
a rare one stays in pieces

Why not just split on spaces?

BY WORD

No bound on the vocabulary. Every surname, typo and compound is a new entry — and anything unseen arrives as a single **<UNK>**.

BY CHARACTER

A tiny vocabulary, and nothing is ever unseen. But sequences get several times longer, against an attention cost that grows with the *square* of length.

BY MERGED PAIR

Bounded vocabulary, no **<UNK>** ever, sequence length close to word-level. You pay in rare words, which stay in pieces.

WHAT THAT COSTS YOU

Whitespace and casing change the split, so your prompt's *formatting* has a price — and non-English text usually costs more tokens for the same meaning.

From words to vectors

SPARSE · ONE-HOT

One dimension per word. `cat` and `kitten` are exactly as far apart as `cat` and `houses` — the representation encodes no meaning at all, and it grows with the vocabulary.

DENSE · DISTRIBUTED

A few hundred continuous dimensions, fixed no matter how large the vocabulary. Similar meanings land near each other, so distance becomes comparable to meaning.

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

The comparison operator divides the lengths out and keeps only the angle. Direction is meaning; magnitude is not.

What one-hot actually looks like

The obvious encoding: one dimension per word in the vocabulary, a single **1**, zeros everywhere else.

One dimension per word

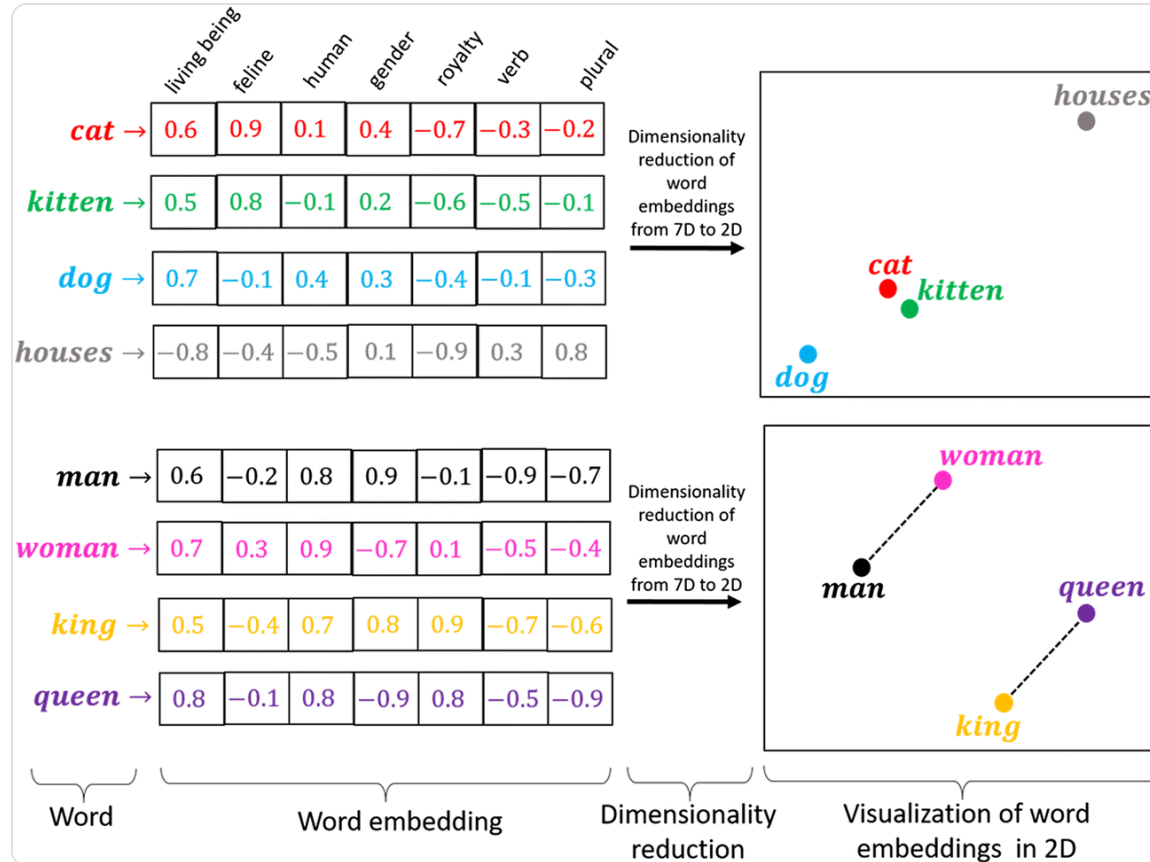
the eight dimensions

cat	1	0	0	0	0	0	0
kitten	0	1	0	0	0	0	0
dog	0	0	1	0	0	0	0
houses	0	0	0	1	0	0	0
man	0	0	0	0	1	0	0
woman	0	0	0	0	0	1	0
king	0	0	0	0	0	0	1
queen	0	0	0	0	0	0	1

8 words → 8 dimensions. 50,000 words → 50,000 dimensions.

every distinct pair of words is exactly as far apart as every other pair

What the space looks like



THE NUMBERS ARE THE MEANING

Each word is a few hundred coordinates. No single one is "felineness" — the meaning is spread across all of them.

NEIGHBOURHOODS ARE SEMANTIC

Project them down and **cat** lands beside **kitten**, while **houses** sits far away.

SO ARE THE DIRECTIONS

The step from **man** to **woman** is roughly the step from **king** to **queen**. Retrieval, in Lecture 06, is built entirely on this.

Static vs contextual

STATIC

One vector per word, forever. "Bank" has a single vector covering the river and the money.

CONTEXTUAL

The vector depends on the sentence. "Bank" by a river and "bank" holding your savings get different vectors.

A transformer produces contextual representations — that is most of what the next section is about.

The Transformer

Mechanism, not derivation

We are taking two steps at a time



WHAT WE ARE SKIPPING, AND WHY

You have just done embeddings. The next step in this picture is not one we are climbing.

DEEP LEARNING FUNDAMENTALS

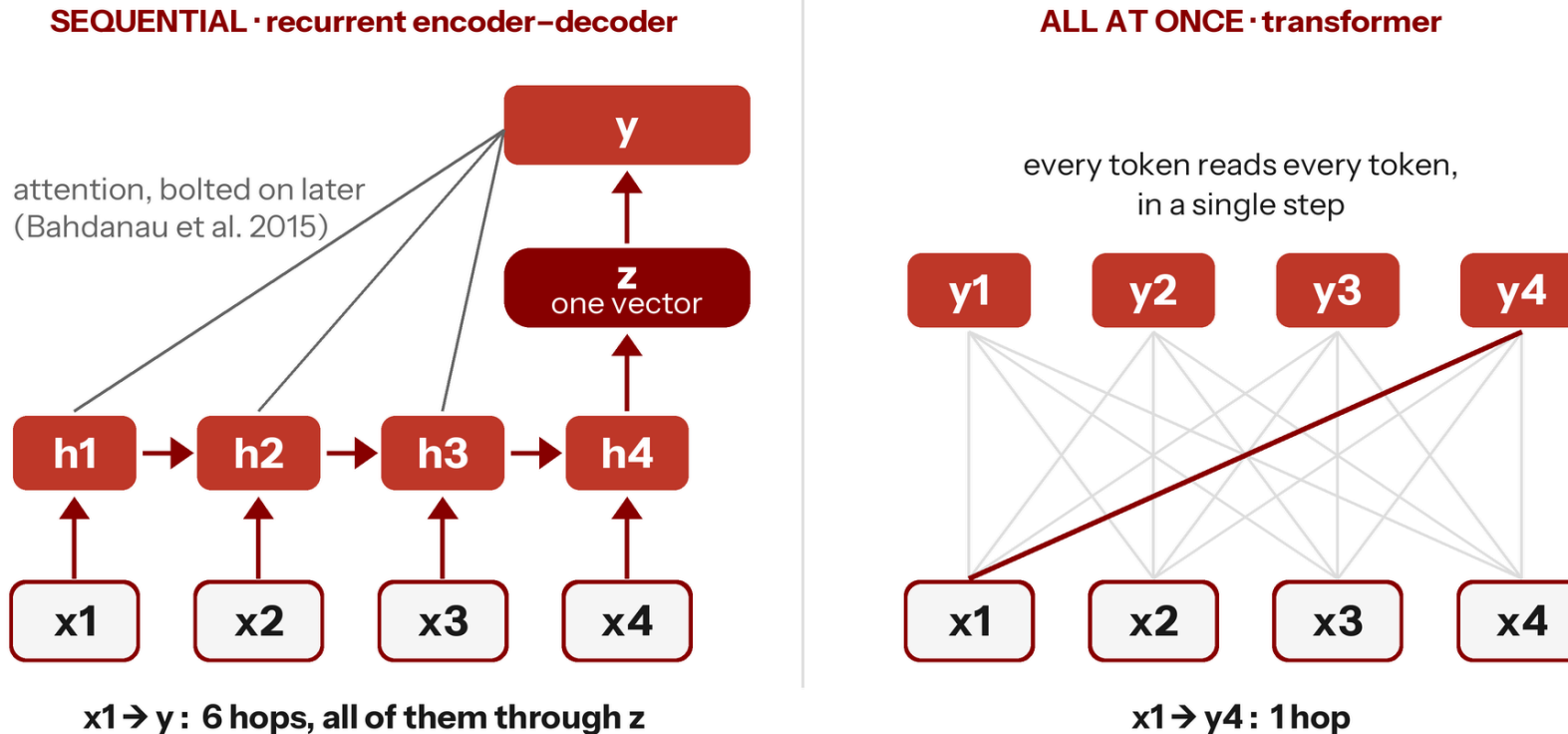
HOW A RECURRENT NETWORK WORKS

HOW ANY OF THIS IS TRAINED

Two shapes

Every token must reach every other token. There are two ways to arrange that.

Two shapes



left: step t needs step $t-1$, so training is strictly ordered. right: every position at once.

DISCUSSION

The transformer replaced the recurrent network almost completely. What did it buy — and what did it pay?

What it bought, and what it cost

BOUGHT · PARALLELISM

Step t no longer needs step $t-1$, so the whole sequence trains at once. **This, not accuracy, is why transformers won** — it is what made training on the internet possible.

BOUGHT · NO BOTTLENECK

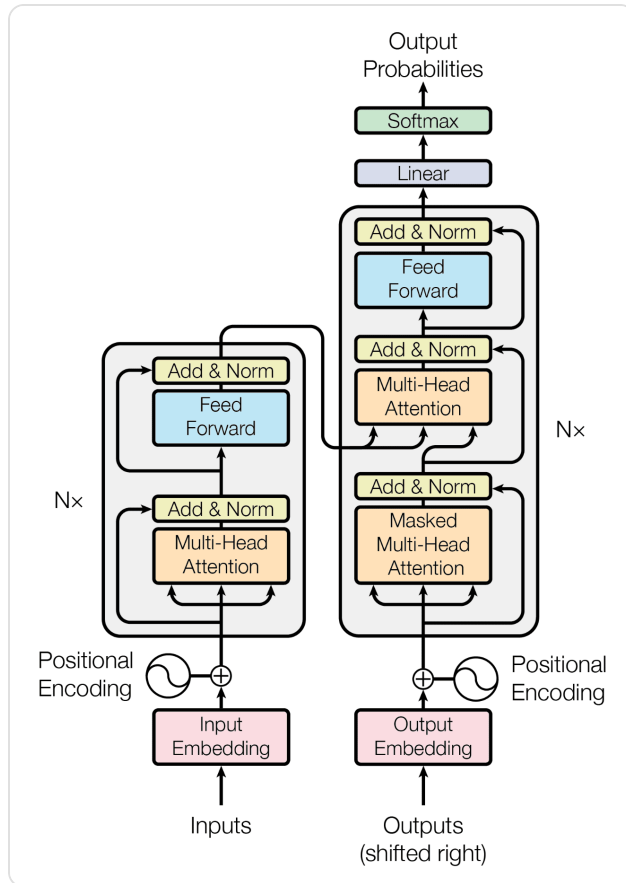
Nothing is squeezed through a single vector, and any two positions are one hop apart rather than many. Distance stops degrading the signal.

PAID · MEMORY

The recurrent model carried **$O(1)$** state per step. The transformer pays **$O(n^2)$** attention and keeps a cache that grows with every token it generates.

It traded memory for speed. Whether that trade holds up under real traffic is the entire subject of **Lecture 03**.

The architecture, as published



ONLY TWO KINDS OF BLOCK

Multi-Head Attention and **Feed Forward**, wrapped in *Add & Norm*.
Everything else on this page is plumbing.

N× MEANS N×

The stack is one block repeated. Depth is the only thing that changes between a small model and a large one.

ENDS IN A SOFTMAX

Top right: a distribution over the vocabulary. That is the whole output.

VASWANI ET AL. 2017 · FIGURE 1

The operation, stated

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

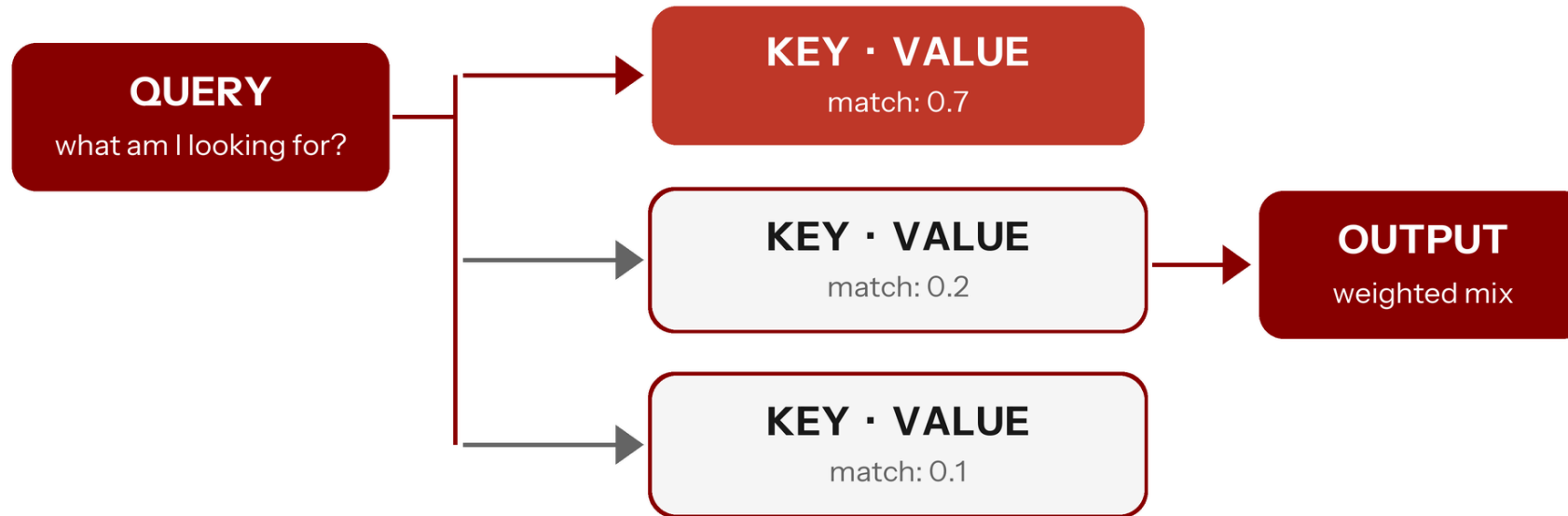
The same three lines you just read, in symbols. We are not going to derive it.

- ▶ **$QK^\top$** — score every query against every key, all pairs at once.
- ▶ **$\div \sqrt{d_k}$** — keep the scores in a range where softmax has not saturated.
- ▶ **$\text{softmax}(\dots) V$** — turn scores into weights that sum to 1, then mix the values by them.

Attention as a soft lookup

Each token emits a **query**, and offers a **key** and a **value**.

every token asks a question; every token offers an answer



the **KEYS** and **VALUES** of earlier tokens never change

which is why they are worth caching — see Lecture 03

multi-head attention runs several of these lookups in parallel

Reading that diagram

THE THREE ROLES

- **Query** — what this token is looking for
- **Key** — what a token advertises
- **Value** — what it passes on if matched

WHY "SOFT"

- A hash lookup returns *one* entry
- This returns a blend of *all* of them
- Nothing is ever fully selected or fully discarded

Which raises an obvious question: what do those weights actually look like?

What that looks like on a real sentence

Rows sum to 1. Columns do not. The upper half does not exist.

Self-attention weights

row = the token attending · column = the token attended to

	Hello	I	love	you	row sum
Hello	1.00	—	—	—	1.00
I	0.30	0.70	—	—	1.00
love	0.15	0.25	0.60	—	1.00
you	0.10	0.15	0.35	0.40	1.00

the grey half is masked — a token can never attend to one that comes after it

every row sums to 1 · no column does · attention is directed, not symmetric

Multi-head, position, and depth

MULTI-HEAD

Several lookups in parallel.
Each head is free to track a different relationship — syntax, coreference, topic.

POSITIONAL ENCODING

Attention is order-blind: it sees a *set*. Position has to be injected, or "dog bites man" reads like "man bites dog".

DEPTH

Attention → feed-forward, with residuals and normalisation, stacked N times. Later layers work on what earlier ones assembled.

Almost none of that survived unchanged

The figure you just looked at is from 2017. Here is what the model you will actually call has done to it.

2017	Today	What it bought
encoder + decoder	decoder-only	one stack — the half you actually call
sinusoidal position	RoPE (rotary)	relative position, and it extends past training length
post-LayerNorm	pre-norm + RMSNorm	trains deep without a warm-up schedule
ReLU feed-forward	SwiGLU	better quality at the same parameter count
multi-head attention	grouped-query (GQA)	heads share K and V, so the cache shrinks
dense feed-forward	mixture of experts	parameters and compute stop being one number

The last two rows are cost, not capability — and cost is **Lecture 03**.

Parallel to train, sequential to generate

TRAINING

Every position is known in advance, so the whole sequence is processed at once. This is why transformers scale.

GENERATION

Token $n+1$ depends on token n . No parallelism is available across the sequence — you wait for each one.

That asymmetry is the entire subject of Lecture 03.

Where we got to

1· It never received your text — it received tokens, chosen by a merge table it learned before it ever saw you.

2· Attention let every token read every earlier token — in one step, with the upper triangle masked so nothing reads the future.

3· And now it is about to emit exactly one thing — which is where session 2 starts, and where everything you can actually control lives.

The whole objective, in one line

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

The probability of an entire document — an essay, a proof, a working program — is a product of local next-token decisions. There is no term in it for a plan.

EVERY STEP

- One score per vocabulary token
- Softmax into a distribution
- Pick one, append it, repeat

WHAT FOLLOWS

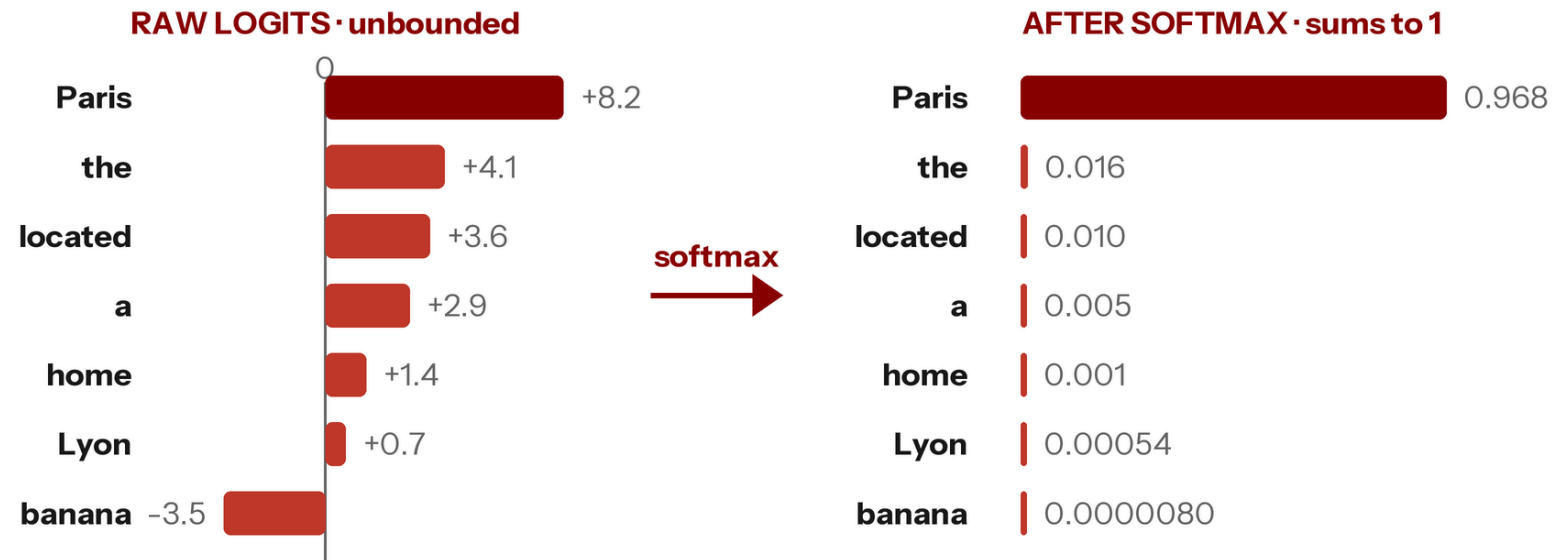
- Long-range structure is emergent, not planned
- The distribution is always well-formed — even when the model is wrong

What actually comes out

Not a word, and not a probability — one **unbounded score** per vocabulary token.

One score per token, every step

context: "The capital of France is"



...and 49,993 more. Every one of them scored, every step.

sample one · append it · run the whole thing again

DISCUSSION

The model just gave you a confident, entirely invented citation.

Is that a bug in the training data? And could you fix it by adding *"say I don't know if you are unsure"* to the system prompt?

Nothing in the objective asks whether it is true

$$P(x_1 \dots x_n) = \prod_t P(x_t \mid x_{<t})$$

The whole training objective. Look for the term that measures truth.

THERE IS NO SUCH TERM

The objective rewards the **likely** next token, not the correct one. A plausible-looking citation is exactly what a high-likelihood continuation of "according to" looks like.

SO THE PROMPT CANNOT FIX IT

"Say I don't know" asks the model to report a confidence it never computed. It changes the *style* of the output, not what the model knows.

Why hallucination is structural

THE UNCOMFORTABLE PART

A model that always produces a next-token distribution will always produce fluent text — whether or not it has grounds for it. There is no internal "I don't know" state unless one was trained in.

SO IT IS NOT A BUG

Fluency and factuality are separate properties.
The training objective optimises the first.

WHERE IT IS HANDLED

Detection, grounding and abstention are engineering, and belong to Lecture 13.

Generation Parameters

Four knobs and what each one cuts

Greedy or sampled

GREEDY

Always take the highest-probability token.
Repeatable, and prone to flat, looping text.

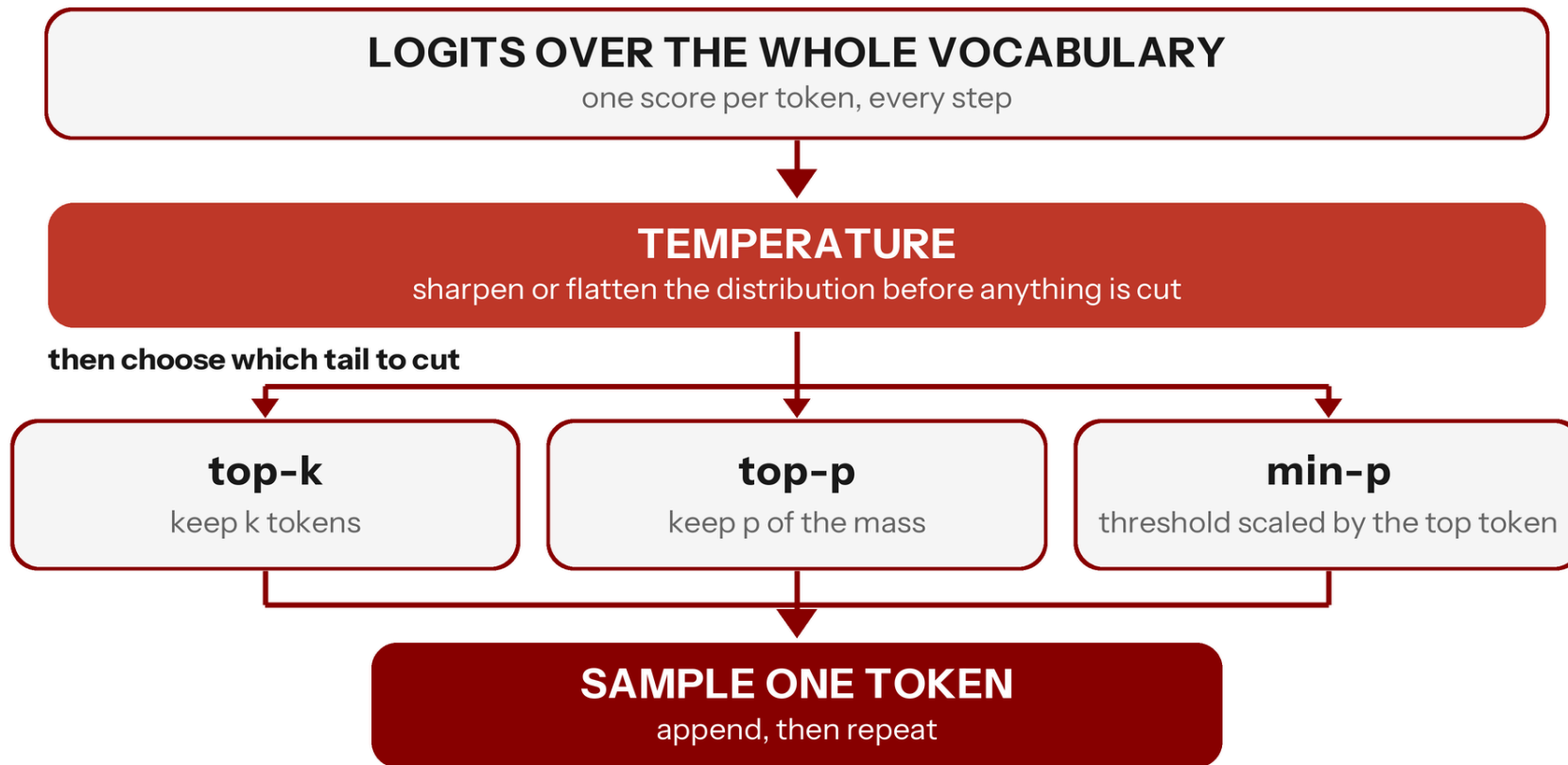
SAMPLING

Draw from the distribution. More varied, more surprising, and more able to be wrong.

Everything below is about **which part of the distribution you are willing to draw from.**

The pipeline

Temperature reshapes; the truncation knobs cut; then one token is drawn.



temperature 0 is near-deterministic — not a guarantee

Temperature

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

The same softmax as the output layer, with one extra division. **T** divides the logits before the exponential — so it can reshape the distribution, but it can never remove a token from it.

LOW · 0.0–0.3

Sharpens toward the top token. Extraction, classification, tool arguments.

MIDDLE · ~0.7

The usual default for conversation.

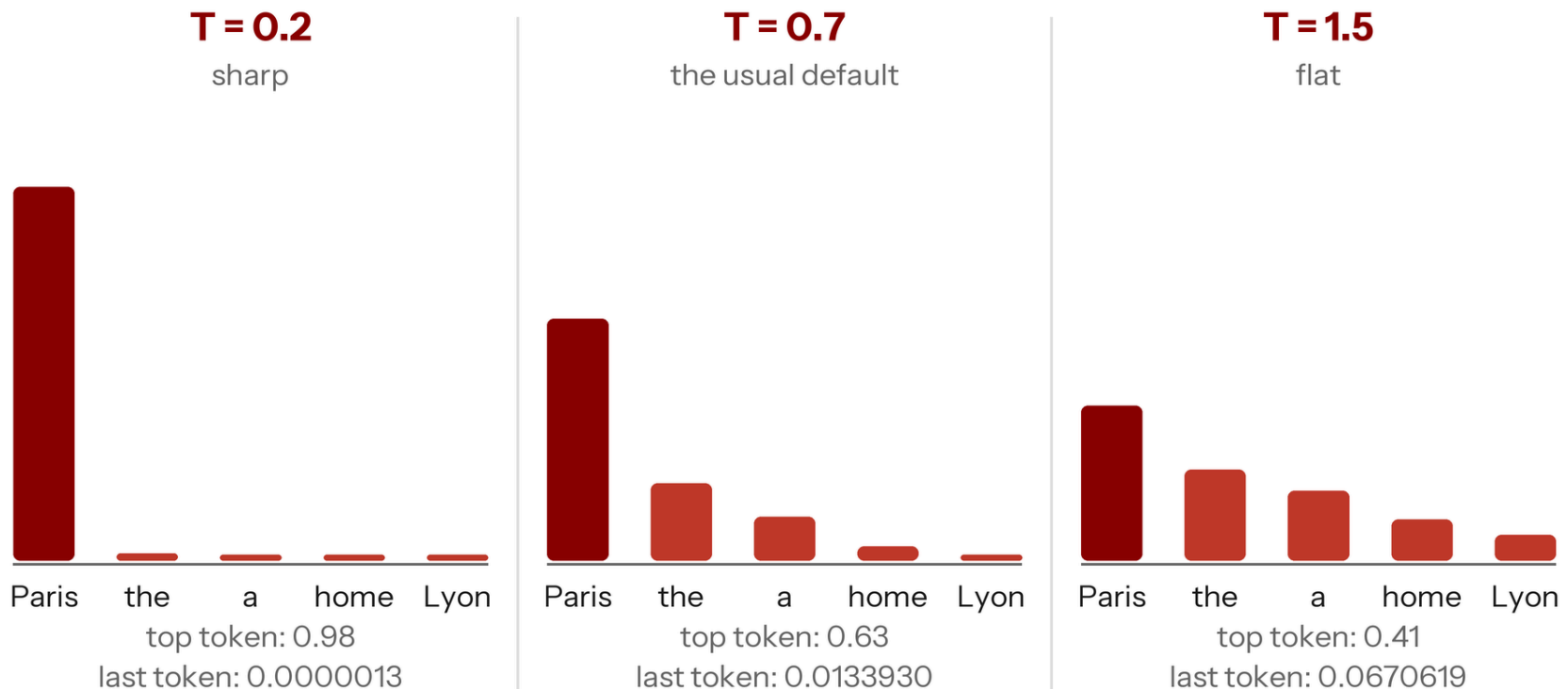
HIGH · >1.0

Flattens it. More variety, more chances to pick something implausible.

The same logits, three temperatures

Look at the short bars, not the tall one. At $T = 0.2$ the tail is crushed — but it is still **there**.

Same logits, three temperatures



every token is still in every panel — temperature reshapes, it never removes

Which tail do you cut?

TOP-K

Keep the **k** most likely tokens.
Fixed count, regardless of how confident the model is.

TOP-P

Keep the smallest set whose probability adds up to **p**.
Adapts: few tokens when confident, many when not.

MIN-P

Keep tokens above a threshold **scaled by the top token's** probability.

PRACTICAL

Set one truncation knob, not three. top-p is the common default; combining them silently makes the strictest one win.

Penalties, and repetition

FREQUENCY PENALTY

Reduces a token's score in proportion to how often it has already appeared. The more it repeats, the harder it is pushed down.

PRESENCE PENALTY

Reduces it once the token has appeared at all, regardless of count. A flat tax on reuse.

Both fight looping. Both, pushed too far, make the model avoid words it genuinely needs — including the ones in your schema.

DISCUSSION

You set **temperature = 0**. Do you now get byte-identical output on every run?

If not — the sampler is no longer choosing anything, so what is left that could still move?

Defaults worth starting from

TOOL ARGUMENTS, EXTRACTION, CLASSIFICATION

Temperature **0**. You want the same answer every time, and creativity is a defect here.

EXPLANATION, DRAFTING, BRAINSTORMING

Temperature **0.7** with top-p **0.9**. Vary one knob at a time when tuning.

Thinking Modes

What the reasoning knob actually does

"Reasoning" is a token-generation pattern

A reasoning model does not run a different algorithm. It runs the loop you just saw, and spends tokens on the problem **before it spends them on the answer.**

Which works for exactly the reason the previous section predicts: every token is conditioned on everything already emitted, so tokens spent laying the problem out are tokens the answer is conditioned on.

Consequence 3, from the start of the lecture: it cannot un-say a token, so anything it must weigh has to be generated first. "Thinking" is that observation, productised.

The knob, and what it buys

OPENAI

`reasoning_effort`

- `none` · `minimal`
- `low` · `medium` · `high`

ANTHROPIC

`output_config.effort`

- `low` · `medium` · `high`
- `xhigh` · `max`

GOOGLE

`thinking_level`

- `MINIMAL` upward
- replaced an integer budget

THE PART WORTH REMEMBERING

All three began as an **integer budget of thinking tokens** and moved to named effort levels. Anthropic deprecated `budget_tokens`; Google deprecated `thinking_budget`. The knob was always a token count — the label just got friendlier.

The bill you never see



REASONING TOKENS ARE OUTPUT TOKENS

They are billed as output, they consume the same context window, and latency scales with them roughly linearly. You usually never see them.

SO THE WINDOW IS NOT WHAT YOU THINK

A trivial exchange can spend a serious fraction of the budget before the first visible word arrives.

THE KNOB DOES NOT BUY THOUGHT

It buys tokens. The same loop, just more of it, up front.

Where the rest of this lives

NOT IN THIS LECTURE

- Prompting *for* reasoning → **Lecture 10**
- Does more effort help? → **Lecture 12**
- How these models are trained → out of scope

WHAT YOU TAKE FROM HERE

- The knob is a token budget wearing a friendlier label
- Effort is a cost decision before it is a quality one
- Everything above is still one loop, one token at a time

It generates. It does not think.

You have now watched every step of it.

The questions, answered

Why it miscounts letters — it never received letters. "strawberry" arrived as subword pieces.

Why the same prompt gives different answers — it samples. Temperature and truncation decide how much of the tail is in play.

Why a long document costs more — cost and the context limit are both denominated in tokens, not words.

And it still is not thinking — you now know what it does instead, and therefore where you get to intervene.

Summary

COVERED

- BPE, and tokens as the unit of context and cost
- Sparse vs dense vectors, and cosine similarity
- What the transformer replaced, and its price
- Attention as a soft lookup — directed, masked
- Logits, softmax, and structural hallucination
- Temperature, truncation, penalties, determinism
- Reasoning effort as a knob over token count

KEEP THESE

- It cannot see what tokenization removed
- Direction is meaning; length is not
- Keys and values never change — cache them
- Fluent is not the same as correct
- One truncation knob, set deliberately
- The arithmetic is exact; the shape is not
- It generates. It does not think.

References · representation and architecture

- Vaswani et al. 2017 · **Attention Is All You Need** · arXiv 1706.03762 — architecture figure reproduced from Figure 1
- Sennrich et al. 2016 · **Neural Machine Translation of Rare Words with Subword Units** · arXiv 1508.07909
- Mikolov et al. 2013 · **Efficient Estimation of Word Representations in Vector Space** · arXiv 1301.3781
- Sutskever et al. 2014 · **Sequence to Sequence Learning with Neural Networks** · arXiv 1409.3215
- Bahdanau et al. 2015 · **Neural Machine Translation by Jointly Learning to Align and Translate** · arXiv 1409.0473
- Su et al. 2021 · **RoFormer: Enhanced Transformer with Rotary Position Embedding** · arXiv 2104.09864
- Zhang & Sennrich 2019 · **Root Mean Square Layer Normalization** · arXiv 1910.07467
- Xiong et al. 2020 · **On Layer Normalization in the Transformer Architecture** · arXiv 2002.04745
- Shazeer 2020 · **GLU Variants Improve Transformer** · arXiv 2002.05202

References · generation and behaviour

- Shazeer 2019 · **Fast Transformer Decoding: One Write-Head is All You Need** · arXiv 1911.02150
- Ainslie et al. 2023 · **GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints** · arXiv 2305.13245
- Fedus et al. 2021 · **Switch Transformers** · arXiv 2101.03961
- Holtzman et al. 2020 · **The Curious Case of Neural Text Degeneration** · arXiv 1904.09751
- Thinking Machines Lab 2025 · **Defeating Nondeterminism in LLM Inference** · thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/
- OpenAI · **Reasoning models** · developers.openai.com/api/docs/guides/reasoning
- Anthropic · **Effort and adaptive thinking** · platform.claude.com/docs/en/build-with-claude/effort
- Google · **Gemini thinking** · ai.google.dev/gemini-api/docs/thinking

What's Next · Lecture 03: LLM Inference & Serving

WHAT'S COVERED

- Prefill vs decode, and why they behave differently
- The KV cache — and what it costs in memory
- Static vs continuous batching; PagedAttention
- Constrained decoding as a mechanism

HOW TODAY CONNECTS

- You saw the masked triangle — now: why cache K and V?
- Tokens as the unit becomes tokens as the cost
- Sequential generation becomes a systems problem

Thank You

Questions & Discussion

Lecture 02 · LLM Agent Development